

Lab 3: Spatial Data Downloading (USGS EarthExplorer + Google Earth Engine + geemap)

Goal: Practice downloading data from USGS EarthExplorer and geemap

Estimated time: 120-180 minutes **Last updated:** 2026-02-02

Learning outcomes

- Find and download geospatial data from a web portal (USGS EarthExplorer) using an Area of Interest (AOI) and filters.
- Set up Google Earth Engine (GEE) access and confirm your account is working.
- Use geemap in JupyterLab to load Earth Engine imagery, compute NDVI, and export a result.
- Practice “data inventory” skills using NumPy/Pandas (small table you can reuse in your final project).

What to submit (required screenshots)

ID	Screenshot content
S1	EarthExplorer: Search Criteria page showing your AOI + date range.
S2	EarthExplorer: Data Sets page showing the dataset(s) you selected (e.g., Landsat Collection 2 Level-2).
S3	EarthExplorer: Results list showing at least one scene selected (include cloud cover/preview).
S4	EarthExplorer: Download Options page (or your browser downloads list) showing the file is downloading or downloaded.
S5	GEE access: Code Editor opens successfully OR a confirmation that your account is enabled (any clear proof).
S6	JupyterLab: geemap map showing ROI + an imagery layer (e.g., Sentinel-2 RGB).
S7	JupyterLab: NDVI layer visible with a colorbar/legend.
S8	JupyterLab: NDVI data export to google drive (Optional)

Part A — Download data using USGS EarthExplorer (dashboard workflow)

Goal: practice “portal downloading” with clear filters and a documented dataset choice.

A1. Create an account and sign in

1. Open USGS EarthExplorer in a web browser and sign in (create an account if you do not have one).
2. Confirm you can reach the EarthExplorer search interface after login.

A2. Set your search area (AOI) and time range

3. Go to the Search Criteria tab.
4. Choose an AOI method: (a) draw a polygon/rectangle on the map, (b) type coordinates, or (c) upload a boundary file.
5. Set a Date Range appropriate for your topic (for practice, use a 1–6 month window).
6. Optional: if available, set a maximum Cloud Cover filter (e.g., ≤ 20 –40%) to reduce unusable scenes.

Tip

Pick an AOI you can describe in one sentence (e.g., “Charlotte metro + buffer”).
Small AOIs return faster and make later demos easier.

A3. Select datasets (what to choose for this lab)

7. Go to the Data Sets tab.
8. For optical satellite imagery, select Landsat Collection 2 Level-2 (surface reflectance) as your primary dataset.
9. Optional (extra practice): also select an elevation dataset (e.g., SRTM/DEM) to see how non-imagery products appear in the portal.

Why Level-2?

- Level-2 products are already corrected and typically easier to analyze than raw products.
- Surface reflectance is a common starting point for vegetation indices and land-surface analysis.

A4. Review results and choose a scene

10. Go to the Results tab.
11. Sort/filter results by Cloud Cover (lower is usually better).
12. Open the preview/footprint for 1–2 candidate scenes and confirm your AOI is covered.
13. Select at least one scene to download (you can download more than one if you want).

A5. Download and organize your files

14. Click Download Options (or the download icon) for your selected scene.
15. Choose the main product package for the dataset you selected and start the download.
16. Create a local folder structure like: data/raw/usgs/landsat/ and move the downloaded file(s) there.
17. Keep the original filename (do not rename yet).

Part B — Set up Google Earth Engine (GEE)

Goal: make sure you can authenticate and run a simple Earth Engine command from Python.

Tip

Install code formatter to save time

```
conda install -c conda-forge jupyterlab_code_formatter black isort
```

B1. One-time account setup

1. Use a Google account you will keep for the semester.
2. Request/enable Earth Engine access for that account (if you have never used GEE before).
3. Open the Earth Engine Code Editor and confirm it loads without permission errors.

B2. Python authentication check (JupyterLab)

Run the following in a Jupyter notebook (see Part C for environment setup):

```
import ee, geemap
geemap.ee_initialize()

# quick sanity check (should print a CRS string like 'EPSG:4326' or similar)
img = ee.Image('USGS/SRTMGL1_003')
print(img.projection().crs().getInfo())
```

If you see a permissions or “not enabled” error

- Make sure you are authenticated with the same Google account you enabled for Earth Engine.
- Try re-running `geemap.ee_initialize()` and follow the link/code authentication flow.
- If the message mentions a Cloud project/API, create a Google Cloud project and enable the Earth Engine API, then retry.

Part C — Download data via Earth Engine using geemap

Goal: access imagery without manual downloads and create a simple analysis-ready product (NDVI).

C1. Environment + JupyterLab

1. Activate your course conda environment (e.g., `conda activate geoviz`).
2. Install geemap if needed: `conda install -c conda-forge geemap`
3. Launch JupyterLab: `jupyter lab`

C1b. Warm-up: verify NumPy/Pandas and create a mini data inventory

This is practice for the final project README. You do NOT submit the output.

```
import numpy as np
import pandas as pd

print("numpy:", np.__version__)
print("pandas:", pd.__version__)

inventory = pd.DataFrame([
    {"source": "USGS EarthExplorer", "dataset": "Landsat C2 L2 (SR)", "resolution": "30 m", "license": "check metadata", "notes": "downloaded 1 scene"},
    {"source": "GEE (via geemap)", "dataset": "Sentinel-2 SR Harmonized", "resolution": "10-20 m", "license": "check collection docs", "notes": "NDVI + time series"},
])

inventory
```

C2. geemap end-to-end demo code (copy/paste/format)

```

import ee, geemap
from geemap import chart

geemap.ee_initialize()

# 1) ROI (edit if you want)
roi = ee.Geometry.Rectangle([-81.05, 35.10, -80.60, 35.45])

# 2) Map
m = geemap.Map(center=[35.2271, -80.8431], zoom=10)
m.add_basemap("HYBRID")
m.addLayer(roi, {"color": "yellow"}, "ROI", False)

# 3) Sentinel-2 SR (cloud-masked) with timestamp preserved
def mask_s2_sr(img):
    scl = img.select("SCL")
    mask = (scl.neq(3)
            .And(scl.neq(8))
            .And(scl.neq(9))
            .And(scl.neq(10))
            .And(scl.neq(11)))
    out = img.updateMask(mask).divide(10000)
    return out.copyProperties(img, img.propertyNames())

s2 = (ee.ImageCollection("COPERNICUS/S2_SR_HARMONIZED")
      .filterBounds(roi)
      .filterDate("2026-01-01", "2026-01-31")
      .map(mask_s2_sr)
      .filter(ee.Filter.notNull(["system:time_start"])))

s2_median = s2.median().clip(roi)
m.addLayer(s2_median, {"min": 0, "max": 0.3, "bands": ["B4", "B3", "B2"]}, "Sentinel-2 RGB")

# 4) NDVI
ndvi = s2_median.normalizedDifference(["B8", "B4"]).rename("NDVI")
ndvi_vis = {"min": 0, "max": 1, "palette":
["#440154", "#3b528b", "#21918c", "#5ec962", "#fde725"]}
m.addLayer(ndvi, ndvi_vis, "NDVI")
m.add_colorbar(ndvi_vis, label="NDVI", layer_name="NDVI")

m

```

C3. Optional: export NDVI to Google Drive

```

geemap.ee_export_image_to_drive(
    image=ndvi,
    description="ndvi_demo_geovis",
    folder="geemap_exports",
    region=roi,
    scale=20,
    maxPixels=1e13,
)
print("Export task submitted. Check the Earth Engine Tasks tab.")

```

Troubleshooting

Problem	Fix
No module named geemap	Install geemap in the SAME conda environment: conda install -c conda-forge geemap
Map widget does not display in JupyterLab	Ensure ipywidgets support: conda install -c conda-forge ipywidgets jupyterlab_widgets
EEException: not signed up / permission denied	Confirm Earth Engine is enabled for your Google account; re-run geemap.ee_initialize().
Error: system:time_start missing	Use copyProperties(...) as shown; also filter notNull(['system:time_start']).